

COMP1324 Algorithmics

Dynamic Programming 4

Hikmat Farhat

Subset sum

- We are given an array A of n elements and an integer value S and we ask if there is a subset of A whose sum is S
- For example

$$A = [3, 5, 4, 8] \quad S = 9$$

- We start with the last element. There are two choices
 - 1 Either the last element is in the sum in which case we have to find a subset of the first $n - 1$ elements whose sum is $S - A[n]$
 - 2 Or the last element is not in the sum in which case we have to find a subset of the first $n - 1$ elements whose sum is S

Subset sum

Algorithm 1: Subset sum

input: sum S and array of elements A with $A[0]$ unused.

Result: Returns true if there is a subset of A whose sum is S

function SubsetSum(A, n, S)

if $S = 0$ **then**

return true

Why?

if $n = 0$ **then**

return false

return SubsetSum($A, n - 1, S$) || SubsetSum($A, n - 1, S - A[n]$)

Example

- Consider the array $A = [2, 7, 3, 1]$ and $S = 6$.

$$\begin{aligned} \text{subset}(A, 4, 6) &= \text{subset}(A, 3, 6) \vee \text{subset}(A, 3, 5) \\ &= \text{false} \vee \text{true} = \text{true} \end{aligned}$$

$$\begin{aligned} \text{subset}(A, 3, 6) &= \text{subset}(A, 2, 6) \vee \text{subset}(A, 2, 3) \\ &= \text{false} \vee \text{false} = \text{false} \end{aligned}$$

$$\begin{aligned} \text{subset}(A, 3, 5) &= \text{subset}(A, 2, 5) \vee \text{subset}(A, 2, 2) \\ &= \text{false} \vee \text{true} = \text{true} \end{aligned}$$

$$\text{subset}(A, 2, 6) = \text{false}$$

$$\text{subset}(A, 2, 3) = \text{false}$$

$$\text{subset}(A, 2, 5) = \text{false}$$

$$\text{subset}(A, 2, 2) = \text{true}$$

Bottom up

Algorithm 2: Bottom up solution for Subset Sum

input: sum S and array A of size $n + 1$ with $A[0]$ unused.

Result: Returns matrix $opt[i][j] = true$ iff there is a subset of $A[1 : i]$ whose sum is j

function SubsetSum(A, S)

```
    for  $i = 0$  to  $n$  do
        |  $opt[i][0] \leftarrow true$ 
    for  $j = 1$  to  $S$  do
        |  $opt[0][j] \leftarrow false$ 
    for  $i = 1$  to  $n$  do
        | for  $j = 1$  to  $S$  do
            | |  $opt[i][j] \leftarrow opt[i - 1][j] \vee opt[i - 1][j - A[i]]$ 
    return  $opt$ 
```

Subset sum: example

Example: Consider the set $=\{2, 8, 3, 13, 7\}$ and target sum of 12
The bottom up solution produces the following matrix:

	0	1	2	3	4	5	6	7	8	9	10	11	12
1	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	1	0	1	0	0	0
1	0	1	1	0	1	0	0	1	0	1	1	0	0
1	0	1	1	0	1	0	0	1	0	1	1	0	0
1	0	1	1	0	1	0	1	1	1	1	1	1	①

The result we are seeking is in the last row and last column of the matrix. Since it is 1 the answer is true: the set contains a subset whose sum is 12.

Subset sum: finding the solution

- $A[i]$ is in the solution only if
 - 1 $s - A[i] \geq 0$ where s is the "remaining" sum
 - 2 $opt[i - 1][s - A[i]] = true$
- The above leads to the following algorithm (assuming $opt[n][S] = true$)

```
function SSsolution(A,S,opt)
```

```
     $i, val \leftarrow n, S$ 
```

```
     $sol \leftarrow \emptyset$ 
```

```
    while  $i \neq 0 \wedge val \neq 0$  do
```

```
        if  $val - A[i] \geq 0 \wedge opt[i - 1, val - A[i]]$  then
```

```
             $sol \leftarrow sol \cup A[i]$ 
```

```
             $val \leftarrow val - A[i]$ 
```

```
             $i \leftarrow i - 1$ 
```

```
    return  $sol$ 
```

Finding the solution: example

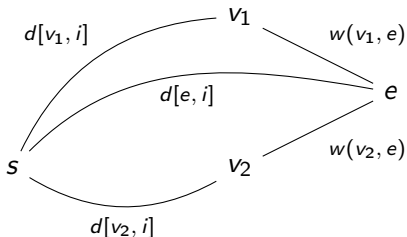
In the previous example where the set $= \{2, 8, 3, 13, 7\}$ and target sum of 12 we can recover the solution by using solution matrix:

	0	1	2	3	4	5	6	7	8	9	10	11	12
0	①	0	0	0	0	0	0	0	0	0	0	0	0
1	1	0	①	0	0	0	0	0	0	0	0	0	0
2	1	0	①	0	0	0	0	0	1	0	1	0	0
3	1	0	1	1	0	①	0	0	1	0	1	1	0
4	1	0	1	1	0	①	0	0	1	0	1	1	0
5	1	0	1	1	0	1	0	1	1	1	1	1	①

Thus the solution is $\{2, 3, 7\}$

Bellman-Ford Single Source Shortest Path

- We have already seen a version of Bellman-Ford where the edges are repeatedly relaxed ($n - 1$ times)
- Here we give a dynamic programming version of Bellman-Ford
- Let $G = \langle V, E, w \rangle$ be weighted graph.
- The quantity to be optimized is the shortest distance from source s to some destination e using **at most i edges**.
- We will find the optimal substructure with the aid of the example below.



Optimal substructure

- In the figure there are three paths from s to e .
 - ① A path with i edges from s to e .
 - ② A path with i edges from s to v_1 followed by edge (v_1, e)
 - ③ A path with i edges from s to v_2 followed by edge (v_2, e)
- The shortest path from s to e using *at most* $i + 1$ edges is the minimum of the three paths.
- In general we obtain the Bellman equation:

$$d[u, i + 1] = \min \left(d[u, i], \min_{v \in V} (d[v, i] + w(v, u)) \right)$$

- With the help of the above recurrence and the fact that if a shortest path exists (no negative cycles) then it uses at most $n - 1$ edges (why?) we can solve the single source shortest path problem.
- Note that we can detect negative cycles by running the algorithm an extra iteration (why?).

Bellman-Ford Algorithm

Algorithm 3: Bellman-Ford Algorithm

input: vertex set V , array of reverse adjacency lists $inDeg$

Result: Returns shortest path distance matrix d from source s

function Bellman-Ford($V, inDeg$)

```
    foreach  $v \in V$  do
        |  $d[0][v] \leftarrow \infty$ 
     $d[0][s] \leftarrow 0$ 
    for  $i \leftarrow 1$  to  $|V|$  do
        | foreach  $v \in V$  do
            | |  $t \leftarrow \min_{u \in inDeg(v)} (d[i-1][u] + w(u, v))$ 
            | |  $d[i][v] \leftarrow \min(d[i-1][v], t)$ 
    return  $d$ 
```

How do we compute $inDeg$? Time complexity?

Complexity

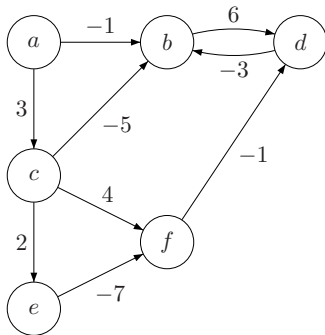
Algorithm 4: Bellman-Ford Algorithm

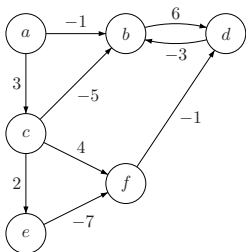
function Bellman-Ford($V, inDeg$) **foreach** $v \in V$ **do** $d[0][v] \leftarrow \infty$ $d[0][s] \leftarrow 0$ **for** $i \leftarrow 1$ **to** $|V|$ **do** **foreach** $v \in V$ **do** $t \leftarrow \min_{u \in inDeg(v)} (d[i-1][u] + w(u, v))$ $d[i][v] \leftarrow \min(d[i-1][v], t)$ **return** d } $\Theta(n)$ $\Theta(n)$ times} $\Theta(m)$

The total complexity is $\Theta(nm)$

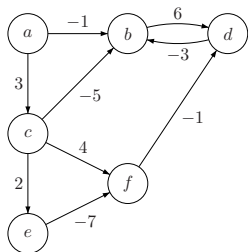
Example

In the graph below we would like to use the Bellman-Ford algorithm to compute the shortest path from node a to all other nodes.

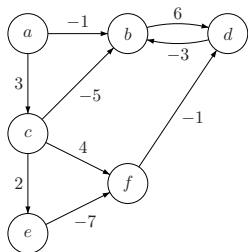




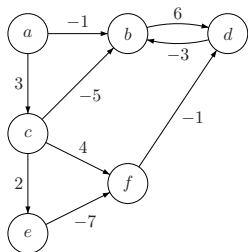
Dest \ iter	0	1	2	3	4	5
a	0	0	0	0	0	0
b	∞					
c	∞					
d	∞					
e	∞					
f	∞					



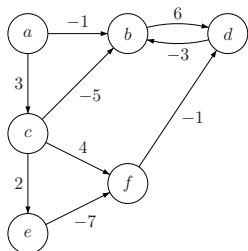
Dest \ iter	0	1	2	3	4	5
a	0	0	0	0	0	0
b	∞	-1				
c	∞	3				
d	∞	∞				
e	∞	∞				
f	∞	∞				



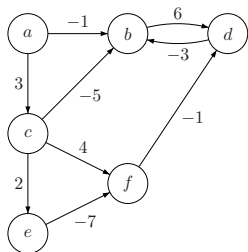
Dest \ iter	0	1	2	3	4	5
a	0	0	0	0	0	0
b	∞	-1	-2			
c	∞	3	3			
d	∞	∞	5			
e	∞	∞	5			
f	∞	∞	7			



Dest \ iter	0	1	2	3	4	5
a	0	0	0	0	0	0
b	∞	-1	-2	-2		
c	∞	3	3	3		
d	∞	∞	5	4		
e	∞	∞	5	5		
f	∞	∞	7	-2		



Dest \ iter	0	1	2	3	4	5
a	0	0	0	0	0	0
b	∞	-1	-2	-2	-2	
c	∞	3	3	3	3	
d	∞	∞	5	4	-3	
e	∞	∞	5	5	5	
f	∞	∞	7	-2	-2	



Dest \ iter	0	1	2	3	4	5
a	0	0	0	0	0	0
b	∞	-1	-2	-2	-2	-6
c	∞	3	3	3	3	3
d	∞	∞	5	4	-3	-3
e	∞	∞	5	5	5	5
f	∞	∞	7	-2	-2	-2

Floyd-Warshall

- The Floyd-Warshall algorithm is used to find the shortest path between **all pairs** of vertices in a weighted graph.
- It uses the properties of **intermediate** vertices of paths.
- Let $G = \langle V, E, w \rangle$ be a weighted graph with $V = \{v_1, \dots, v_n\}$.
- Given a simple path $p = \langle v_1, \dots, v_k \rangle$ an **intermediate** vertex is any vertex of p other than v_1 and v_k .
- Denote by Π_{ij}^k be the set of paths from v_i to v_j whose **intermediate** vertices are drawn from the set $\{v_1, \dots, v_k\}$.

Example: intermediate vertices

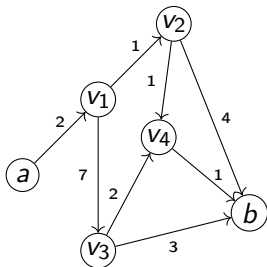
- Note that in the example below the shortest path from a to b , π , using Π_{ab}^3 is the same as using Π_{ab}^2 but different from the one using Π_{ab}^4 .

$$\Pi_{ab}^1 = \emptyset$$

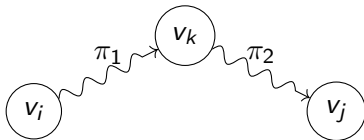
$$\Pi_{ab}^2 = a \rightarrow v_1 \rightarrow v_2 \rightarrow b$$

$$\Pi_{ab}^3 = \Pi_{ab}^2 \cup \{a \rightarrow v_1 \rightarrow v_3 \rightarrow b\}$$

$$\begin{aligned} \Pi_{ab}^4 = \Pi_{ab}^3 \cup \\ \{a \rightarrow v_1 \rightarrow v_3 \rightarrow v_4 \rightarrow b\} \cup \\ \{a \rightarrow v_1 \rightarrow v_2 \rightarrow v_4 \rightarrow b\} \end{aligned}$$



General case



- Let $\pi \in \Pi_{ij}^k$ be a path with the smallest weight.
- There are two cases:
 - 1 Either $v_k \notin \pi$ in which case $\pi \in \Pi_{ij}^{k-1}$
 - 2 Or $v_k \in \pi$ then π can be decomposed into two (shortest) paths π_1 and π_2 where $\pi_1 \in \Pi_{ik}^{k-1}$ and $\pi_2 \in \Pi_{kj}^{k-1}$ (see figure).
- The second case uses the fact that for any shortest path $v_i, \dots, v_k, \dots, v_l, \dots, v_j$ the subpath $v_k, \dots, v_l$ is a shortest path from v_k to v_l for all k, l . (why?).

Floyd-Warshall

- Let d_{ij}^k be the weight of the shortest path from v_i to v_j whose intermediate vertices are drawn from the set $\{v_1, \dots, v_k\}$.
- The previous analysis leads to the following recurrence relation:

$$d_{ij}^{(k)} = \begin{cases} w(i, j) & \text{if } k = 0 \\ \min \left(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \right) & \text{if } k > 0 \end{cases}$$

Bottom up algorithm

Algorithm 5: Floyd-Warshall Algorithm

input: Weights matrix W

Result: Returns the shortest path distance between all pairs, $d_{ij}^{(n)}$, as a matrix $D^{(n)}$

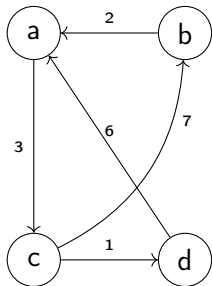
function Floyd-Warshall(W)

```
 $D^{(0)} \leftarrow W$ 
for  $k \leftarrow 1$  to  $n$  do
    for  $i \leftarrow 1$  to  $n$  do
        for  $j \leftarrow 1$  to  $n$  do
             $d_{ij}^{(k)} \leftarrow \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$ 
return  $D^{(n)}$ 
```

Example: Floyd-Warshall

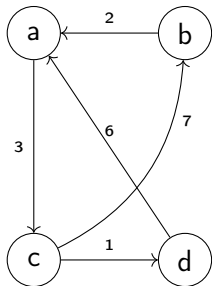
using no node

$$d^{(0)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{pmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & \infty & \infty \\ \infty & 7 & 0 & 1 \\ 6 & \infty & \infty & 0 \end{pmatrix} \end{matrix}$$



Example: Floyd-Warshall

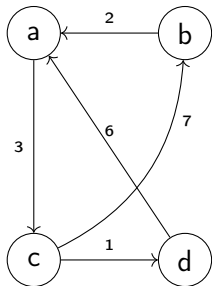
using only $\{a\}$



$$d^{(0)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{pmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & \infty & \infty \\ \infty & 7 & 0 & 1 \\ 6 & \infty & \infty & 0 \end{pmatrix} \end{matrix}$$

$$d^{(1)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{pmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & 5 & \infty \\ \infty & 7 & 0 & 1 \\ 6 & \infty & \infty & 0 \end{pmatrix} \end{matrix}$$

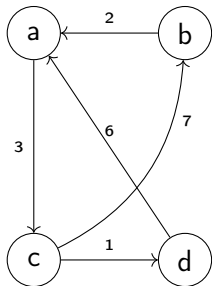
Example: Floyd-Warshall

using only $\{a\}$ 

$$d^{(0)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{pmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & \infty & \infty \\ \infty & 7 & 0 & 1 \\ 6 & \infty & \infty & 0 \end{pmatrix} \end{matrix}$$

$$d^{(1)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ \text{red } d \end{matrix} & \begin{pmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & 5 & \infty \\ \infty & 7 & 0 & 1 \\ 6 & \infty & 9 & 0 \end{pmatrix} \end{matrix}$$

Example: Floyd-Warshall

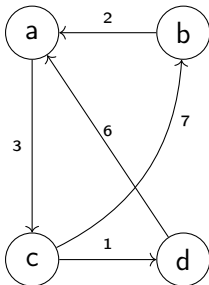
using only $\{a\}$ 

$$d^{(0)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{pmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & \infty & \infty \\ \infty & 7 & 0 & 1 \\ 6 & \infty & \infty & 0 \end{pmatrix} \end{matrix}$$

$$d^{(1)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{pmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & 5 & \infty \\ \infty & 7 & 0 & 1 \\ 6 & \infty & 9 & 0 \end{pmatrix} \end{matrix}$$

Example: Floyd-Warshall

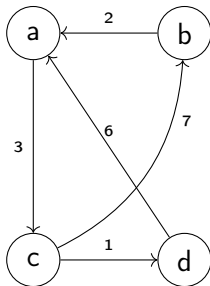
using only $\{a, b\}$



$$d^{(1)} = \begin{matrix} & a & b & c & d \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{pmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & 5 & \infty \\ \infty & 7 & 0 & 1 \\ 6 & \infty & 9 & 0 \end{pmatrix} \end{matrix}$$

$$d^{(2)} = \begin{matrix} & a & b & c & d \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{pmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & 5 & \infty \\ 9 & 7 & 0 & 1 \\ 6 & \infty & 9 & 0 \end{pmatrix} \end{matrix}$$

Example: Floyd-Warshall

using only $\{a, b, c\}$ 

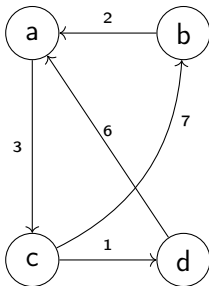
$$d^{(2)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{pmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & 5 & \infty \\ 9 & 7 & 0 & 1 \\ 6 & \infty & 9 & 0 \end{pmatrix} \end{matrix}$$

$$d^{(3)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} \rightarrow a \\ b \\ c \\ d \end{matrix} & \begin{pmatrix} 0 & 10 & 3 & \infty \\ 2 & 0 & 5 & \infty \\ 9 & 7 & 0 & 1 \\ 6 & \infty & 9 & 0 \end{pmatrix} \end{matrix}$$

Example: Floyd-Warshall

using only $\{a, b, c\}$

$$d^{(2)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{pmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & 5 & \infty \\ 9 & 7 & 0 & 1 \\ 6 & \infty & 9 & 0 \end{pmatrix} \end{matrix}$$

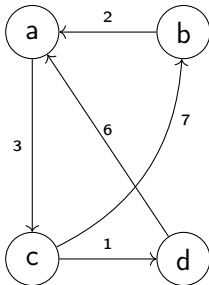


$$\rightarrow d^{(3)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{pmatrix} 0 & 10 & 3 & 4 \\ 2 & 0 & 5 & \infty \\ 9 & 7 & 0 & 1 \\ 6 & \infty & 9 & 0 \end{pmatrix} \end{matrix}$$

Example: Floyd-Warshall

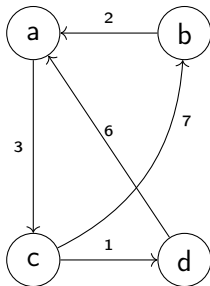
using only $\{a, b, c\}$

$$d^{(2)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{pmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & 5 & \infty \\ 9 & 7 & 0 & 1 \\ 6 & \infty & 9 & 0 \end{pmatrix} \end{matrix}$$



$$d^{(3)} \Rightarrow \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{pmatrix} 0 & 10 & 3 & 4 \\ 2 & 0 & 5 & 6 \\ 9 & 7 & 0 & 1 \\ 6 & \infty & 9 & 0 \end{pmatrix} \end{matrix}$$

Example: Floyd-Warshall

using only $\{a, b, c\}$ 

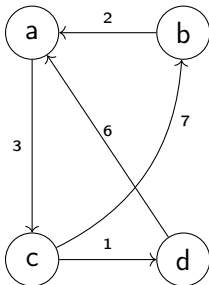
$$d^{(2)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ \mathbf{c} \\ d \end{matrix} & \begin{pmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & 5 & \infty \\ 9 & \mathbf{7} & 0 & 1 \\ 6 & \infty & \mathbf{9} & 0 \end{pmatrix} \end{matrix}$$

$$d^{(3)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ \rightarrow d \end{matrix} & \begin{pmatrix} 0 & 10 & 3 & 4 \\ 2 & 0 & 5 & 6 \\ 9 & 7 & 0 & 1 \\ 6 & \mathbf{16} & 9 & 0 \end{pmatrix} \end{matrix}$$

Example: Floyd-Warshall

using $\{a, b, c, d\}$

$$d^{(3)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{pmatrix} 0 & 10 & 3 & 4 \\ 2 & 0 & 5 & 6 \\ 9 & 7 & 0 & 1 \\ 6 & 16 & 9 & 0 \end{pmatrix} \end{matrix}$$



$$d^{(4)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{pmatrix} 0 & 10 & 3 & 4 \\ 2 & 0 & 5 & 6 \\ 7 & 7 & 0 & 1 \\ 6 & 16 & 9 & 0 \end{pmatrix} \end{matrix}$$

→

References

- The Floyd-Warshall algorithm is covered in section 23.2 of your [textbook](#) and section 18.4 of the Algorithms Illuminated [book](#)
- The Bellman-Ford algorithm using DP is covered in section 18.2 of the Algorithms Illuminated [book](#)
- Note that we have already covered the Bellman-Ford algorithm using a non DP approach in the "Shortest Paths" lecture on graphs. It is covered extensively in section 22.1 of your [textbook](#)
- A Python implementation of the subset sum problem can be found [here](#), for Bellman-Ford [here](#), for Floyd-Warshall [here](#)